

CO4 – AT1: DATA INTERPRETATION

Name: Archishman | Reg. No: 192425341

Q1. Real-Time Detection System

Method	Accuracy	Speed
Viola-Jones	78%	Fast
YOLO	92%	Moderate
Deep Learning	95%	Slow

Comparison

Viola-Jones offers the fastest processing but the lowest accuracy (78%), making it prone to missed or false detections. Deep Learning delivers the highest accuracy (95%) but its slow speed makes it unsuitable where results are needed instantly. YOLO sits in between, combining high accuracy (92%) with moderate speed, achieved through a single-pass detection architecture that processes the full image in one forward run instead of scanning it region by region.

Evaluation & Decision

For a surveillance system operating under strict latency constraints, YOLO is the most suitable choice. It sacrifices only 3% accuracy compared to Deep Learning while offering substantially faster inference, which is critical for real-time frame-by-frame monitoring. Viola-Jones, though fastest, is not reliable enough for security-critical detection, and Deep Learning's slow speed risks delayed or missed alerts during live monitoring.

Justification (Deployment Conditions)

- Real-time surveillance requires continuous frame processing (often 15–30 FPS); Deep Learning cannot keep pace at scale.
- YOLO's 92% accuracy is sufficient for practical threat/object detection while keeping latency low.
- YOLO also generalizes well on edge devices/GPUs commonly used in surveillance hardware, unlike heavier Deep Learning pipelines.

Q2. Robust Object Detection

Method	Normal	Occlusion	Low Light
Viola-Jones	80%	60%	55%
YOLO	90%	75%	70%
Deep Learning	95%	85%	80%

Comparison

Across all three conditions, Deep Learning consistently outperforms YOLO and Viola-Jones, maintaining accuracy above 80% even in low light. Viola-Jones deteriorates sharply under occlusion and low light (falling to 55–60%) because it relies on fixed Haar-like features that are highly sensitive to lighting and partial visibility. YOLO degrades more gracefully, retaining 70–75% accuracy under adverse conditions due to its learned convolutional feature representations.

Evaluation & Trade-offs

Deep Learning provides the best overall robustness and is the preferred method when accuracy under difficult, uncontrolled conditions is the top priority — for example, in forensic analysis or offline video review. However, this robustness comes at the cost of higher computational load and slower processing (as seen in Q1), so it is less suited to real-time constraints.

- Viola-Jones: fastest but least robust; usable only in controlled, well-lit, unobstructed environments.
- YOLO: balanced robustness and speed; the practical choice for real-time deployments with variable conditions.
- Deep Learning: highest robustness; best where accuracy outweighs speed, e.g., post-event analysis.

Q3. Optical Flow Analysis

Comparison

- Method A: Produces stable, smooth motion vectors but fails to capture small/subtle motions — favors precision and consistency over sensitivity.
- Method B: Captures fine, small-scale motion but the resulting vectors are noisy — favors sensitivity over stability.

Evaluation & Justification

For motion tracking in dynamic scenes, Method A is generally more suitable. Dynamic scenes already involve large, rapid movements that need to be tracked reliably across frames; stability and consistency of motion vectors are more important here than detecting minute movements, since noisy vectors (as in Method B) can cause tracking drift, false trajectories, and instability in subsequent frames.

However, if the application specifically requires detecting subtle or fine-grained motion (e.g., detecting small gestures or vibration analysis), Method B would be preferred despite its noise, provided it is paired with a smoothing/filtering stage (e.g., Kalman filtering) to reduce noise sensitivity.

Conclusion

Overall, Method A is the safer general-purpose choice for dynamic-scene motion tracking due to its superior stability, while Method B is reserved for niche applications needing fine motion detail.

Q4. Model Generalization and Overfitting (10 Marks)

Model	Training Accuracy	Testing Accuracy
A	98%	70%
B	90%	88%

Comparison

Model A shows a large gap between training accuracy (98%) and testing accuracy (70%) — a difference of 28 percentage points. This is a classic sign of overfitting: the model has memorized patterns and noise specific to the training data rather than learning generalizable features. Model B shows a much smaller gap (90% vs 88%, only 2 percentage points), indicating it has learned patterns that generalize well to unseen data.

Generalization Ability & Overfitting Risk

- Model A: High overfitting risk. Excellent on seen data but unreliable on new data — performance will likely degrade significantly in production.
- Model B: Low overfitting risk, strong generalization. Slightly lower peak training accuracy, but consistent performance across both training and testing sets.

Evaluation & Selection

Model B is more reliable for real-world deployment. Real-world data always differs somewhat from training data, so a model's testing/validation accuracy is a far better predictor of production performance than training

accuracy. Model B's small train-test gap indicates it will perform close to its reported 88% accuracy in practice, whereas Model A's real-world performance would likely resemble its 70% testing accuracy or worse, despite the impressive-looking 98% training score.

Q5. System Performance under Constraints

Model	Accuracy	Latency	Cost
A	88%	50 ms	Low
B	92%	80 ms	Moderate
C	95%	120 ms	High

Comparison

Model A offers the lowest latency (50 ms) and lowest cost but the weakest accuracy (88%). Model C achieves the highest accuracy (95%) but at more than double the latency of Model A (120 ms) and the highest cost. Model B sits in the middle on all three metrics: 92% accuracy, 80 ms latency, and moderate cost.

Evaluation & Best Balance

Model B provides the best overall balance for a real-time application. It delivers a substantial accuracy improvement over Model A (+4%) while adding only a moderate latency increase (+30 ms), and its cost remains reasonable compared to Model C. Model C's accuracy gain over Model B (+3%) is comparatively small relative to its steep latency (+40 ms) and cost increase, making it inefficient unless the application is extremely accuracy-critical and latency-tolerant (e.g., offline analytics).

Justification (Deployment Constraints)

- Real-time systems typically have latency budgets (often under 100 ms) — Model C's 120 ms may violate this, while Model B's 80 ms fits comfortably.
- Cost-efficiency favors Model B over Model C for similar performance tiers, especially at scale.
- Model A is best reserved for extremely latency-sensitive, resource-constrained deployments where a small accuracy trade-off is acceptable.

Conclusion: Model B is recommended as it optimally balances accuracy, latency, and cost for practical real-time deployment.